Redrob Ranker v2.0 — "Council of Nine"

Technical Report: Intelligent Candidate Discovery & Ranking

A system that ingests a large pool of candidate profiles (JSON) and a single job description (JD), and returns a ranked, explained shortlist of the most relevant candidates. The engine is CPU-only, fully offline at ranking time, and deterministic. The same engine powers a CLI (`rank.py` , which writes a top-100 `submission.csv`) and an interactive recruiter dashboard ("NextHire": a FastAPI backend in `api/` + a Next.js frontend in `web/`).

This report covers four things in order: **the approach**, **what was built**, **why it was built that way**, and **how it works step by step**. Every claim is grounded in the actual code (real file, function, and variable names are cited).

---

1. Approach

1.1 The overall strategy

The system treats candidate ranking as a classic **two-phase Information Retrieval problem — retrieve, then rank** — wrapped in an **interpretable multi-signal scoring layer** that encodes recruiter judgment that raw text similarity cannot capture.

1. **Interpret the JD once, offline, into structured intent.** The job description is not fed to the model as free text at runtime. It is pre-parsed into a static artifact, `src/jd_intent.json` , that decomposes the role into machine-usable vocabularies: a natural-language `query_text` , `must_have_capabilities` , `nice_to_have` , `positive_titles` , `negative_titles` , `evidence_verbs` , `evidence_domain` , `product_industries` , `services_companies` , `offdomain_skills` , `ir_nlp_skills` , plus disqualifier vocabularies (`research_titles` , `wrapper_skills` , `core_ml_skills` , `leadership_titles` , `eval_skills` , `external_validation_markers`). Each field maps to a specific downstream scorer. Because the JD is already structured, the ranking step needs **no network and no LLM**.
2. **Retrieve a shortlist with hybrid search.** Over the full pool, a hybrid retriever (`src/retrieve.py`) builds two complementary representations — lexical (TF-IDF) and dense/semantic (LSA) — and fuses their rankings with **Reciprocal Rank Fusion** to produce a shortlist of `SHORTLIST_SIZE = 4000` candidates.
3. **Score the shortlist with the "Council of Nine."** Each shortlisted candidate is converted into engineered features (`src/features.py`) and scored by nine interpretable sub-scorers (`src/council.py`). Six are additive and weighted; three act as multiplicative gates (integrity, negative screen, availability), with a fourth gate (JD disqualifier screen) layered on.
4. **Sharpen the head, then calibrate and order.** The top `RERANK_SIZE = 200` are re-scored by a two-stage re-ranker (`src/rerank.py`), then all candidates are assigned a coarse relevance band and a final calibrated score (`src/score.py`), and a grounded natural-language justification is generated per candidate (`src/reasoning.py`).

1.2 How resumes and JDs are understood and compared

A candidate profile is a nested JSON object (see `candidate_schema.json`): `profile` , `career_history[]` , `education[]` , `skills[]` , `certifications[]` , and a rich `redrob_signals` block of behavioral telemetry. Comparison happens in two layers:

Layer A — text similarity. `features.build_document(c)` flattens the honest, evidence-bearing parts of a profile into one lowercase string: headline, summary, current title/industry, then for every role its title, industry, and **description repeated twice** (descriptions are where genuine "built/shipped a system" evidence lives, so they are weighted), then the skills list and education fields. The JD side is its `query_text`. Both are embedded into:

- a **lexical** space — TF-IDF over word unigrams+bigrams, cosine similarity; and
- a **dense/semantic** space — LSA (TruncatedSVD over the TF-IDF matrix) to `DENSE_DIM = 256` dimensions, cosine similarity. (Optionally a sentence-transformers bi-encoder, `BAAI/bge-small-en-v1.5`.)

Layer B — engineered signals. Text similarity alone cannot tell a genuine engineer from a keyword-stuffer, or an active candidate from a ghost. `features.compute_features(c, jd)` derives ~40 numeric/categorical signals — title identity, domain-anchored evidence counts, *verified* skill trust, experience band, tenure stability, product-vs-services ratio, recency, responsiveness, integrity flags, and more — that the council reasons over.

1.3 Which ML/NLP techniques are used, and why those

Technique	Where	Why this one
TF-IDF (sparse lexical), uni+bigrams, `sublinear_tf`, `min_df=2`	`HybridRetriever.fit`	Exact-term/jargon matching ("vector search", "recsys", "learning to rank"); deterministic, zero-download, fast on CPU.
LSA = TruncatedSVD over TF-IDF, 256-dim, L2-normalized	`HybridRetriever.fit`	Captures synonymy/topical similarity beyond exact tokens; a CPU-friendly, offline stand-in for neural embeddings. Fit on a 40k sample, transform all rows.
Reciprocal Rank Fusion (RRF), `RRF_K = 60`	`HybridRetriever.retrieve`	Fuses two heterogeneous signals using *ranks*, not raw scores, so no cross-signal calibration is needed.
Cross-encoder re-ranker (`cross-encoder/ms-marco-MiniLM-L-6-v2`) with a deterministic feature fallback	`src/rerank.py`	A small neural model can re-judge the top-200 query↔profile pairs jointly; falls back to a pure-feature re-rank when weights are absent.
Interpretable rule/heuristic scorers (the Council)	`src/council.py`	Encodes recruiter judgment (keyword-stuffing, ghosting, services-only, research-only) that similarity misses; every score carries a human-readable rationale and is fully auditable.
Optional neural/LTR upgrades (sentence-transformers, ONNX+INT8, XGBoost/LambdaMART, FAISS)	`requirements*.txt`, `onnx_optimize.py`	Documented swap-ins behind identical interfaces, so the pipeline can scale up without a rewrite.

The defaults (TF-IDF + LSA + RRF + rule-based council) were chosen so the system runs reliably **offline, on CPU, deterministically, and at 100k-profile scale**; the neural components are strict upgrades that degrade gracefully to the defaults if their model weights are not present.

1.4 What fundamentally decides relevance

Relevance is a **gated composite**. For each candidate, the six additive sub-scorers are fused into a weighted `core` fit, which is then multiplied by a chain of bounded gates and nudged by small additive bonuses (`src/score.py`, `score_pool`):

```
core            = Σ_k  council_part[k] · weight[k]   /  Σ_k weight[k]
final_fit       = core × integrity × negative_screen × disqualifier_screen
final           = final_fit × availability + soft_nudges
```

A candidate is therefore "more relevant" than another when it shows **more verified, demonstrated, role-aligned capability** (semantic fit + a real engineering title + concrete build/ship evidence + endorsement-backed skills + in-domain specialization + the right experience band) **and** is not knocked down by a plausibility, negative-screen, disqualifier, or availability gate. A final **3-band relevance gate** (`relevance_band`) then guarantees that higher-relevance candidates always sort above lower-relevance ones before scores are calibrated into presentable, strictly non-increasing values.

---

2. What Was Built

2.1 The ranking engine (`redrob-ranker-v2/src/`)

Module	Stage	What it does	In → Out
`config.py`	—	Central, documented configuration: all weights, thresholds, sizes, bounds, seeds. Single source of truth for every knob that influences a decision.	env vars → constants
`load.py`	`[0a]`	Streams the candidate pool. Auto-detects JSONL vs. a JSON array, transparently handles `.gz` , uses `orjson` when available, and skips malformed lines.	file path → `list[dict]`
`features.py`	`[0b]`	`build_document()` makes the retrieval text; `compute_features()` derives the ~40 numeric/categorical signals. `REFERENCE_DATE = 2026-06-13` anchors recency math.	candidate dict, JD → text + feature dict
`retrieve.py`	`[2]`	`HybridRetriever` (TF-IDF + LSA + RRF), `STRetriever` (sentence-transformers dense), and `build_retriever()` which resolves the backend and falls back safely.	docs + JD query → `(shortlist_idx, dense_sim, lexical_sim)`
`precomputed.py`	`[2]`	`PrecomputedRetriever` : a frozen, NumPy-only object that *replays* a cached shortlist + similarities (~1 MB) instead of re-fitting (~220 MB of matrices).	cached arrays → same retrieval contract
`council.py`	`[3]` , `[6b]`	The nine sub-scorers + `disqualifier_screen()` + `deliberate()` which fuses them.	feature dict, semantic sim → scores + rationales
`skills_verify.py`	`[3h]`	`verified_relevant_skills()` (named, endorsement/usage-backed JD skills) and `certification_credibility()` (trusted issuers).	candidate, JD → skill names, cert score
`rerank.py`	`[3b]`	Two-stage head re-rank: cross-encoder when weights exist, else a deterministic `_feature_scores()` fallback; time-guarded.	JD, top-200 head → one relevance score per head row
`integrity.py`	`[6]`	The honeypot / impossibility guard: 4 HARD exclusion rules + soft penalties; `TECH_FIRST_YEAR` table.	candidate → `(integrity_score, is_honeypot, reasons)`
`score.py`	`[3]→[9]`	The orchestrator: `score_pool()` , `finalize_ranking()` , `relevance_band()` , `_assign_banded_scores()` , `_soft_nudge()` . Produces the final ordered, scored records.	candidates, JD, retriever → records + stats

Module	Stage	What it does	In → Out
`reasoning.py`	`[10]`	`generate()` writes a 1–2 sentence, fact-grounded, rank-calibrated justification with deterministic phrasing variety.	candidate, features, scores → string
`fairness.py`	`[7]`	`audit()` measures group representation and the disparate-impact (4/5ths) ratio over `region` and `institution_tier`.	pool, selected indices → fairness report
`compliance.py`	`[9h]`	`write_audit()` writes an immutable, timestamped JSON audit record (input hash, weights, timings, fairness) per run.	run metadata → `compliance/audit_trail/*.json`
`roles.py`	—	A catalogue of 6 role "intent profiles" (the base Senior AI/ML Engineer JD plus Backend, Data Scientist, Frontend, DevOps, Product) sharing the JD schema, for the multi-role dashboard.	role name → JD-shaped dict
`jd_intent.json`	`[1]`	The structured interpretation of the job description — the parsed JD that drives every scorer.	static data

2.2 Top-level scripts (`redrob-ranker-v2/`)

- **`rank.py`** — the single end-to-end entrypoint (`python rank.py --candidates ./candidates.jsonl --out ./submission.csv`). It loads the JD and pool, opportunistically loads the cached retriever (`_load_cached_retriever` , guarded by exact row-count / id-order / query match), runs `score_pool` , runs the fairness audit and writes the compliance record, and writes the output CSV (`candidate_id, rank, score, reasoning`). It forces Hugging Face offline mode *before* any import so an uncached model fails fast instead of hitting the network.
- **`precompute.py`** — the offline half of the two-phase design. Builds the documents, fits the retriever (LSA or sentence-transformers), **freezes** the retrieval result for the fixed JD into `artifacts/` , optionally warms the cross-encoder, and can gzip the index for committing.
- **`onnx_optimize.py`** — optional one-time export of an embedding model or re-ranker to ONNX with INT8 dynamic quantization (a documented CPU-speedup upgrade path; not required to run `rank.py`).

2.3 Frozen artifacts (`redrob-ranker-v2/artifacts/`)

- `retriever.pkl.gz` — the frozen `PrecomputedRetriever` (shortlist + dense similarities) for the JD query.
- `candidate_ids.pkl.gz` — the row-aligned candidate ids the index was built on, used to validate that the cache matches the live pool before it is trusted.

2.4 The API backend (`redrob-ranker-v2/api/`)

- **`main.py`** — the FastAPI app. Exposes `/api/roles` , `/api/rank` , `/api/stage` (chunked, memory-safe file upload), `/api/status` , `/api/leaderboard` , `/api/candidate/{id}` , `/api/analytics` , `/api/compliance` , `/api/honeypots` , `/api/job-intent` , `/api/export` (Excel), task/shortlist CRUD, and the NextAI chat endpoint. When a static Next.js export exists in `web_out/` it is mounted at `/` , so the whole app can run as one process on one port.
- **`ranker.py`** — wraps the `src/` engine for the dashboard with **no logic changes**. Holds the ranked result in memory (`STATE`), runs ranking on a background thread (the UI polls `/api/status`), ranks the **whole pool**

(not just top-100), and derives recruiter-facing views: `summary`, `leaderboard`, per-candidate `detail` + `_candidate_insights` (strengths/weaknesses/risk/ missing-quals/similar-roles), `analytics`, `compliance`, `honeypots`, `job_intent`, `nextai_context`, and `export_excel`. Crucially, it reuses the exact `finalize_ranking` from `score.py`, so dashboard ordering matches the CLI output.

- **`nextai.py`** — a provider-agnostic LLM assistant (OpenAI / Gemini / Anthropic) over the *live* ranking, using only stdlib `urllib`. It sends a compact JSON context and a strict system prompt that forbids fabrication; if no API key is configured it returns a local data snapshot instead.
- **`supabase_store.py`** — a zero-dependency PostgREST client (stdlib `urllib`) that persists each ranking run and its results; reads `.env` by walking up the directory tree.

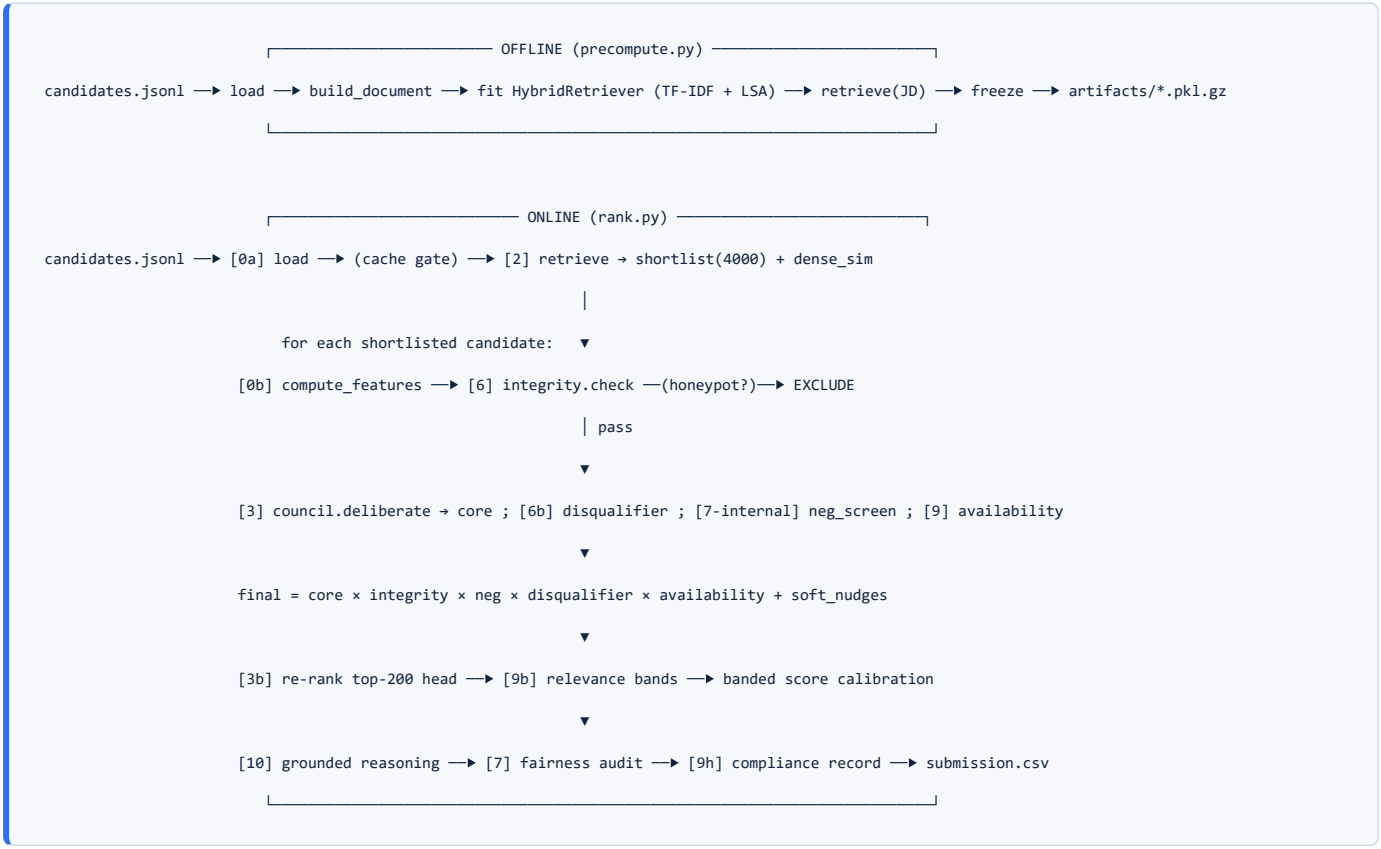
2.5 The web frontend (`redrob-ranker-v2/web/`)

A Next.js + Tailwind single-page dashboard. `app/page.tsx` is the controller: it manages upload → role selection → weight tuning → rank → poll → render, and hosts the tabbed views. `lib/api.ts` is the typed API client; `lib/types.ts` is the shared data model. Components include `Sidebar`, `TopBar`, `Kpis`, `Controls` (council-weight + parameter tuning), `Leaderboard`, `CandidateDrawer`, `InsightsView` (analytics + `charts` / `Donut`), `IntegrityView` (honeypots), `GovernanceView` (fairness/compliance), `CompareView`, `RoleView` (model interpretation of the JD), `PipelineView` (Supabase-backed shortlists), `Logs` (live audit log), and `IngestBanner`.

2.6 Persistence, deployment, and supporting files

- **`supabase_schema.sql`** — four tables: `ranking_tasks`, `task_candidates` (`top200` / `shortlisted` / `honeypot`), `shortlists`, `shortlist_members`.
- **`Dockerfile`** — the ranking-reproduction image (`python:3.13-slim`, designed to run with `--network none`, data mounted at runtime).
- **`Dockerfile.web`** — a two-stage build that statically exports the Next.js UI and serves it from FastAPI as one container on port 7860.
- **`deploy/huggingface/`** — the Space card + Dockerfile that pulls the GHCR image. **`.github/workflows/deploy-web.yml`** builds and pushes that image.
- **`requirements.txt`** (pinned core: numpy, scipy, scikit-learn, orjson, FastAPI, pandas) and **`requirements-embeddings.txt`** (optional sentence-transformers + torch).
- **`compliance/bias_audit_template.json`**, **`submission_metadata.yaml`**.

2.7 The full pipeline architecture (end to end)



The same online path is what `api/ranker.py` runs for the dashboard (over the whole pool), so the CLI and the UI produce identical orderings.

---

3. Why It Was Built That Way

3.1 Why structured JD intent instead of raw text or a runtime LLM

`jd_intent.json` is described in its own header as a "static artifact so the ranking step needs NO network/LLM at run time." Three reasons drive this:

- **Offline determinism.** The ranking step makes zero network calls and yields byte-identical output across runs (`RANDOM_SEED = 13`, fixed `REFERENCE_DATE`).
- **Auditability.** Every signal that influences a hiring decision is a named, inspectable field that maps to a specific scorer — a requirement the code ties explicitly to high-risk hiring-AI documentation obligations.
- **Speed and robustness.** Substring matching against curated vocabularies is cheap and predictable; matching uses `m in name or name in m` so near-misses (e.g. "Recommendation Systems Engineer") still hit.

Trade-off: the JD must be interpreted once up front, and the vocabularies are hand-curated. This is mitigated by `roles.py` (reusable role profiles) and by the fallback logic in features (e.g., domain-anchored evidence falls back to a flat phrase list when the split vocab is absent).

3.2 Why hybrid retrieval + RRF (not a single vector index)

Lexical and dense retrieval fail in opposite ways: TF-IDF misses paraphrase ("ranking systems" vs. "learning to rank"), while a pure embedding misses rare exact tokens (specific tool names like `qdrant`, `bge`, `faiss`). Building both and fusing them by **rank** (`RRF`) gives a recall-safe shortlist without having to make two incomparable score distributions agree. The shortlist of 4000 (`SHORTLIST_SIZE`) is large enough to not lose strong candidates but small enough that the expensive per-candidate scoring stays fast.

3.3 Why LSA by default, with neural models as swap-ins

The default dense signal is LSA (TruncatedSVD on TF-IDF) because it requires **no model download**, runs on CPU, is deterministic, and finishes within a strict runtime budget on a 100k pool. Heavier backends (sentence-transformers bi-encoder for dense, a cross-encoder for head re-ranking, ONNX/INT8 for speed) sit behind the *same* `retrieve()` / `rerank()` contracts and are selected by env vars (`REDROB_EMBED_BACKEND`, `REDROB_RERANK_BACKEND`). Every neural path is wrapped in try/except that falls back to the deterministic default, so a missing dependency or uncached weight never breaks a run — it just downgrades quality gracefully. The cross-encoder default is even set to `"feature"` precisely because its weights are not committed and an `"auto"` load would otherwise stall on a network call.

3.4 Why a frozen `PrecomputedRetriever`

A fully fitted `HybridRetriever` pickles to ~220 MB (the N×V TF-IDF matrix plus the N×256 dense matrix) — too large to commit. But `score_pool` only consumes the `shortlist` and the per-candidate `dense_sim`. So `precompute.py` freezes *just those arrays* (~1 MB) into a NumPy-only `PrecomputedRetriever`, which `rank.py` loads in milliseconds and which produces **byte-identical** results to a live fit. It is deliberately kept in a scikit-learn-free module so unpickling does not drag in scikit-learn. **Safety gates:** the cache is used only if its row count, candidate id/order, and `query_text` all match the live pool; any mismatch triggers a live refit (with reduced "fast-fit" knobs for very large pools to stay within budget).

3.5 Why an interpretable "Council of Nine" instead of one learned model

The council is an ensemble of nine independent sub-scorers, each mapping a principle of human judgment to one measurable feature, each returning a score in `[0,1]` **plus a short rationale**. This was chosen over a single black-box learned ranker for several reasons: it needs **no labeled training data**, every decision is **explainable** (the rationale strings flow straight into `reasoning.py` and the dashboard), and it can encode adversarial-aware judgment (keyword-stuffing, ghosting, services-only, research-only) that a similarity model cannot infer. (A LambdaMART/XGBoost learned fusion is listed as an optional upgrade, not the default.)

The nine, with their rationale and default weights (`config.COUNCIL_WEIGHTS`):

#	Scorer	Signal it measures	Weight / role
1	`semantic_seer`	Dense JD↔profile similarity	0.16
2	`name_rectifier`	Does the title match reality? (anti title-inflation)	0.20
3	`evidence_scout`	Demonstrated "built/shipped a system" evidence	0.22 (highest)
4	`mask_piercer`	Verified skill-trust (anti keyword-stuffer)	0.14
5	`path_reader`	Experience band + tenure stability	0.12
6	`terrain_master`	Product-vs-services + domain proximity	0.16
7	`neti_neti`	Negative screen	multiplier (≥ `0.40`)
8	`integrity`	Plausibility / honeypot	multiplier + hard exclude
9	`availability_oracle`	Behavioral readiness	multiplier `0.55–1.10`

3.6 Why this feature set — the signals that matter for matching

The weights and gates intentionally encode the JD's stated priorities:

- **Career evidence (0.22) and title identity (0.20) dominate**, because the JD asks for people who have *actually shipped* search/ranking/recommendation systems, not people who merely list the words. Evidence is **domain-anchored**: in `compute_features` , a delivery verb (`evidence_verbs`) only counts when it co-occurs with an ML/system domain noun (`evidence_domain`), so generic management language ("led teams", "owned delivery") cannot inflate a non-engineer.
- **The self-declared skills list is deliberately down-weighted (0.14) and *gated by verification*.** A relevant skill contributes `proficiency × verification` , where `verification = max(0.15, 0.5·min(dur/24,1) + 0.3·min(endo/20,1) + 0.2·min(assess/100,1))` — i.e., proficiency only counts to the extent it is backed by usage duration, endorsements, and assessment scores. A wall of "expert" skills with zero endorsements/usage scores near zero.
- **Product-vs-services and in-domain specialization** (`terrain_master`) capture the JD's preference for product-company, NLP/IR-aligned backgrounds over services-firm or off-domain (CV/speech/robotics) careers.
- **Behavioral availability** is a first-class signal because a "perfect on paper but unreachable" candidate is, per the JD, *not actually available*. The `redrob_signals` block (recency, recruiter response rate, open-to-work, interview completion, offer-acceptance, engagement) feeds a bounded modifier.

3.7 Why multiplicative gates (not just additive penalties)

Integrity, negative screen, disqualifier screen, and availability are **multipliers**, not subtractions. This is deliberate:

- A multiplier **bounded and floored** (e.g. `NEGSCREEN_MIN = 0.40`, `DISQUAL_MULT_FLOOR = 0.20`, availability floor `0.55`) can **demote** a candidate sharply without **erasing** genuine merit, so an elite-but-flawed profile is pushed down, not deleted.
- Each gate **requires multiple corroborating signals** before it fires, so a single keyword can never sink a candidate. For example, the `disqualifier_screen` "research-only" rule fires only when research roles are ≥ half the career **and** production evidence is below `EVIDENCE_MIN_FOR_PROD` **and** product ratio is under 0.34.
- The **only hard exclusion** is the integrity honeypot check, and it is tuned to flag *only the logically impossible* — protecting legitimate strong candidates from false positives.

3.8 Why the integrity layer is conservative the way it is

`integrity.py` flags only four impossibilities: (1) ≥3 "expert" skills with 0 months of use, (2) a role whose claimed duration exceeds its real start→end span by a wide margin, (3) a self-contradictory timeline, (4) a skill used longer than its technology has existed (via a conservative `TECH_FIRST_YEAR` table). It **deliberately does not** compare a skill's total usage against the candidate's professional years of experience, because the schema defines skill `duration_months` as total (academic + personal + professional) usage — conflating the two would wrongly exclude, say, an engineer with 7 years of total Elasticsearch use but 4 years of professional tenure. The design bias is "never exclude a real strong candidate."

3.9 Why a final relevance band + calibration

After scoring, `relevance_band()` assigns each candidate to STRONG (2), STANDARD (1), or WEAK (0), and `_assign_banded_scores()` maps each band into a **disjoint score sub-range** (`BAND_RANGES = {2:(0.70,0.99), 1:(0.45,0.69), 0:(0.05,0.44)}`). This guarantees two properties of the output: higher-relevance candidates always outrank lower-relevance ones, and the published score is a clean, strictly non-increasing, presentable number. The final sort key `(-score, candidate_id)` makes ties deterministic (the output validator requires candidate_id-ascending tie-breaking and non-increasing scores by rank).

3.10 Why fairness is audited, not auto-corrected

`fairness.py` measures disparate impact (the 4/5ths rule) on `region` and `institution_tier`, which are in-data attributes, and **logs** it rather than forcibly re-ranking. The stated reasoning: don't silently override genuine merit, and don't infer protected attributes (e.g. gender from names) that would add a noisy, ethically fraught signal. A bounded re-rank nudge is left available for production where legal parity is mandated.

3.11 Why a two-product split (CLI + dashboard)

The CLI exists to produce the canonical artifact (the ranked CSV) reproducibly and offline. The dashboard exists so a recruiter can explore, compare, tune weights, build shortlists, and ask natural-language questions. They share `finalize_ranking` so they never diverge; the dashboard adds an in-memory store, a background ranking thread, optional Supabase persistence, and an optional LLM assistant — all of which are **additive and offline-first** (each degrades to a no-op if not configured).

---

4. How It Works (step by step)

This section traces a single run of `rank.py main()` from raw input to ranked output, naming the actual functions and parameters involved.

Step 0 — Environment and inputs

`rank.py` sets `HF_HUB_OFFLINE` and `TRANSFORMERS_OFFLINE` **before** importing anything, so any neural backend that is uncached fails instantly rather than retrying over the network. It then calls `score.load_jd()` to read `src/jd_intent.json`, and `load.load_candidates(path, limit)` to stream the pool into a `list[dict]`.

Concrete input (from `sample_candidates.json`, `CAND_0000001` — "Ira Vora"): a Backend Engineer at *Mindtree*, 6.9 years' experience, with roles describing "streaming data pipelines on Kafka and Spark", and skills including `NLP` (advanced, 37 endorsements, 26 mo), `Fine-tuning LLMs` (advanced, 21 endorsements, 36 mo), `LoRA` (0 endorsements, 28 mo), plus `Image Classification`, `Speech Recognition`, `TTS`, and `Photoshop`.

Step 1 — Cache gate (fast path)

`_load_cached_retriever(candidates, jd)` looks for `artifacts/retriever.pkl(.gz)`. It returns the frozen `PrecomputedRetriever` **only if** `retr.dense.shape[0]` equals the pool size, `retr.query_text` equals `jd["query_text"]`, and the cached `candidate_ids` exactly equal the live pool's ids in order. Otherwise it returns `None` and the retriever is fit live.

Step 2 — Retrieval → shortlist (`src/retrieve.py`)

If no valid cache, `score_pool` builds documents via `build_document(c)` and calls `build_retriever(docs)`:

- `TfidfVectorizer(max_features=50000, ngram_range=(1,2), min_df=2, sublinear_tf=True)` produces a sparse N×V matrix.
- `TruncatedSVD(n_components=256, n_iter=4, random_state=13)` is **fit on a 40k random sample** (`SVD_FIT_SAMPLE`) and used to **transform all rows**; the result is L2-normalized into `self.dense`. (For pools above `FAST_FIT_POOL_THRESHOLD = 20000`, reduced "fast-fit" knobs keep a live refit within budget.)

Then `retriever.retrieve(jd["query_text"])`:

1. Vectorizes the query into the same lexical and dense spaces (`vectorizer.transform`, then `svd.transform`).
2. `lexical = clip(tfidf @ q_tfidf.T, 0, None)` and `dense = clip(self.dense @ q_dense.T, 0, None)` — cosine similarities.
3. **RRF fusion**, vectorized via an inverse-rank scatter with `RRF_K = 60`:

```
rrf_i = 1/(60 + rank_lexical(i) + 1) + 1/(60 + rank_dense(i) + 1)
shortlist = argsort(-rrf)[:4000]          # SHORTLIST_SIZE
```

It returns `(shortlist, dense_sim, lexical_sim)`. `score_pool` then min-max normalizes `dense_sim` across the shortlist to `[0,1]` for the semantic scorer.

Step 3 — Per-candidate feature extraction (`features.compute_features`)

For each shortlisted candidate, ~40 signals are computed. For `CAND_0000001`:

- **Title:** `current_title` "Backend Engineer" matches `positive_titles` → `cur_title_pos = True`.

- **Company type:** companies are Mindtree, Dunder Mifflin, Mindtree; `services_hits = 2` (Mindtree is in `services_companies`) but not *all* roles, so `services_only = False`; `product_ratio = 0` (industries "IT Services", "Paper Products" are not in `product_industries`).
- **Domain:** `irnlp_hits` ≥ 2 (NLP, Fine-tuning LLMs) and `offdomain_hits` ≥ 4 (Image Classification, Speech Recognition, TTS, Photoshop) → domain proximity ≈ `irnlp/(irnlp+offdomain)` ≈ 0.33.
- **Skill trust:** NLP and Fine-tuning LLMs are endorsement/duration-backed, so they contribute real `relevant_trust`; LoRA (0 endorsements) is a `nice_to_have` and barely counts.
- **Experience:** `yoe = 6.9` lands in the ideal band (`EXP_IDEAL_LOW/HIGH = 6/8`).
- **Behavior:** recency, `recruiter_response_rate`, `open_to_work_flag`, etc., are read from `redrob_signals`.

Step 4 — Integrity gate (`integrity.check`)

Returns `(integrity_score, is_honeypot, reasons)`. If `is_honeypot` is True (any of the four HARD impossibility rules fired), the candidate is **counted and skipped entirely** (`continue` in `score_pool`) — it never reaches the council. Soft inconsistencies instead lower `integrity_score` below 1.0. `CAND_0000001` has no impossibilities, so it passes with `integrity_score = 1.0`.

Step 5 — The Council deliberates (`council.deliberate`)

The six additive scorers run and are fused:

```
semantic_seer   = clip(normalized dense_sim, 0, 1)
name_rectifier  = 1.0 (current title is a genuine engineering role)   # else 0.6 / 0.3 / 0.08
evidence_scout  = min(evidence_hits / 12, 1.0)
mask_piercer    = min(relevant_trust / 4, 1.0)   # capped to 0.35 if many skills but trust < 1.0
path_reader     = band(yoe) × tenure_stability   # band 1.0 in the ideal window
terrain_master  = 0.55·product_ratio + 0.45·domain   # ×0.5 if services_only
core            = Σ part[k]·weight[k] / Σ weight[k]
```

In parallel, the gating scorers produce multipliers:

```
neg_mult           = neti_neti(f)           # floored at 0.40
disqualifier_mult  = disqualifier_screen(f) # floored at 0.20
avail_mult         = availability_oracle(f) # bounded to [0.55, 1.10]
```

`availability_oracle` is a weighted blend mapped into its bounds, with an exponential recency term `recency = exp(-days_inactive/45)` and a **hard ghost gate** (`× AVAIL_HARD_MULT = 0.55`) when a profile is inactive > 150 days **and** answers < 10% of recruiters **and** is not open-to-work.

Step 6 — Compose the per-candidate score (`score_pool`)

```
final_fit = dec["core"] * integ[0] * dec["neg_mult"] * dec["disqualifier_mult"]
final     = final_fit * dec["avail_mult"] + _soft_nudge(f)
final     = max(0.0, final)
```

`_soft_nudge(f)` adds small, bounded additive lifts that live *outside* the normalized core so they nudge but never dominate: location match (+0.025), short notice (+0.015) / long notice (−0.015), a `nice_to_have` trust

bonus (≤ `NICE_TO_HAVE_BONUS_MAX = 0.05`), and an evaluation-rigor bonus for candidates who demonstrate ranking-evaluation/A-B-testing literacy (≤ `EVAL_RIGOR_BONUS_MAX = 0.04`).

Step 7 — Head re-rank (`finalize_ranking` → `rerank.rerank`)

The scored list is sorted by `(-raw, candidate_id)` and each record's `order` is initialized to its `raw` score. The top `RERANK_SIZE = 200` form the **head**. `rerank.rerank(jd, head)` returns one relevance score per head row:

- If a cross-encoder is available, it scores `[query, build_document(candidate)]` pairs jointly (`cross-encoder/ms-marco-MiniLM-L-6-v2`), guarded by `RERANK_TIME_BUDGET_S`.
- Otherwise `_feature_scores()` runs a deterministic re-rank emphasizing `0.34·semantic + 0.30·evidence + 0.22·mask_piercer + 0.14·name_rectifier`, multiplied by the disqualifier/negative gates and a bounded availability nudge.

The head's re-rank scores are min-max mapped back **into the head's own raw range**, so the head reorders internally but never crosses below the (un-re-ranked) tail.

Step 8 — Relevance bands + calibration (`finalize_ranking`)

Every record is assigned `band = relevance_band(f, dec, integ)`:

- **WEAK (0)** if strongly disqualified (`disqualifier_mult ≤ 0.5`), or a non-engineering current title with no engineering role ever, or essentially no relevant signal.
- **STRONG (2)** if verified relevant depth (`relevant_trust ≥ 1.5`) **and** demonstrated delivery (`evidence_hits ≥ 4`) **and** an engineering identity, not services-dominated, not gated.
- **STANDARD (1)** otherwise.

Records are sorted by `(-band, -order, candidate_id)`, the top `TOP_N = 100` are kept, and `_assign_banded_scores()` distributes each band's members across its disjoint sub-range by intra-band min-max on `order`. A final sort by `(-score, candidate_id)` yields the published order. (`CAND_0000001`, with a real engineering title and some verified NLP depth but a services/ paper-products, data-engineering-leaning, partly off-domain profile, lands as a mid-tier STANDARD match rather than STRONG.)

Step 9 — Grounded reasoning (`reasoning.generate`)

For each of the top 100, a 1–2 sentence justification is built that: states specific facts (years, current title, *only* endorsement/usage-verified skills via `verified_relevant_skills`), names the single most decisive positive signal (the highest council part's rationale), honestly surfaces a concern (disqualifier / integrity / non-engineering title / services-only / inactivity / job-hopping / long notice), and matches tone to rank band via `_confidence_word` (top picks read "high-confidence fit"; the bottom band reads "marginal fit"). Phrasing variety is deterministic — seeded by `candidate_id` — so the same candidate always reads the same way but neighbors differ.

Step 10 — Audit and output

Back in `rank.py`:

- `fairness.audit(candidates, stats["selected_idx"])` computes the disparate-impact report over region and institution tier.
- `compliance.write_audit(...)` writes a timestamped JSON record (input fingerprint, candidate/honeypot counts, runtime, the exact council weights and gate thresholds, and the fairness report) into `compliance/audit_trail/`.

- `write_csv(records, out)` emits the final file with the header `candidate_id, rank, score, reasoning`, the score formatted to four decimals, ranks `1..100`, and scores strictly non-increasing with candidate_id-ascending tie-breaks.

Step 11 — The same engine, interactively

In the dashboard, `api/ranker._do_rank` runs this identical path over the whole pool on a background thread, applying any UI-tuned `COUNCIL_WEIGHTS`, experience bands, and notice preference, and toggling the integrity/availability/disqualifier gates on or off. It then exposes the result through the API (`leaderboard`, `detail`, `analytics`, `compliance`, `honeypots`, `job_intent`, Excel `export`) and optionally persists it to Supabase and answers natural-language questions about it via `nextai.chat`.

Appendix — Key constants (from `src/config.py`)

Constant	Value	Meaning
`SHORTLIST_SIZE`	4000	candidates kept after hybrid retrieval
`RERANK_SIZE`	200	head size re-scored by the re-ranker
`DENSE_DIM`	256	LSA dimension for the dense signal
`TFIDF_MAX_FEATURES` / `TFIDF_NGRAM`	50000 / (1,2)	lexical vocabulary cap / n-gram range
`SVD_FIT_SAMPLE` / `SVD_N_ITER`	40000 / 4	LSA fit sample size / iterations
`RRF_K`	60	Reciprocal Rank Fusion constant
`COUNCIL_WEIGHTS`	seer .16, name .20, evidence .22, mask .14, path .12, terrain .16	additive fusion weights
`AVAIL_MIN` / `AVAIL_MAX`	0.55 / 1.10	availability modifier bounds
`NEGSCREEN_MIN`	0.40	negative-screen floor
`DISQUAL_MULT_FLOOR`	0.20	disqualifier floor
`EXP_IDEAL_LOW/HIGH`, `EXP_OK_LOW/HIGH`	6–8, 5–9	experience bands
`BAND_RANGES`	{2:(.70,.99), 1:(.45,.69), 0:(.05,.44)}	disjoint output score sub-bands
`TOP_N`	100	size of the final ranked list
`RANDOM_SEED`	13	determinism